

Acceleration of Heavy Nodes in ONNX Models

Technical Demonstration Report

Executive Summary

This document describes a methodology for identifying and accelerating computationally intensive nodes within ONNX neural network models. Using the onnx2c compiler and gprof profiling tools, this demonstration provides a systematic approach to optimize TinyML model performance by targeting specific operations for hardware acceleration.

1. Context and Motivation

Modern edge AI applications require efficient execution of neural network models on resource-constrained devices. Identifying bottleneck operations within these models is crucial for designing targeted hardware accelerators. This demonstration establishes a workflow for profiling ONNX models to determine which nodes consume the most computational resources and would benefit most from acceleration.

The approach leverages onnx2c to convert ONNX models into C code, enabling standard profiling tools to analyze performance characteristics. This methodology supports the development of application-specific accelerators by providing quantitative data on computational hotspots.

2. Platform and Tool Description

2.1 Implementation Features

The demonstration platform consists of the following components:

- onnx2c compiler for converting ONNX models to C implementations
- gprof profiling tool for performance analysis
- Python preprocessing pipeline for dataset preparation
- Benchmark harness for model execution and measurement

2.2 Data Flow and Processing

The workflow processes data through the following stages:

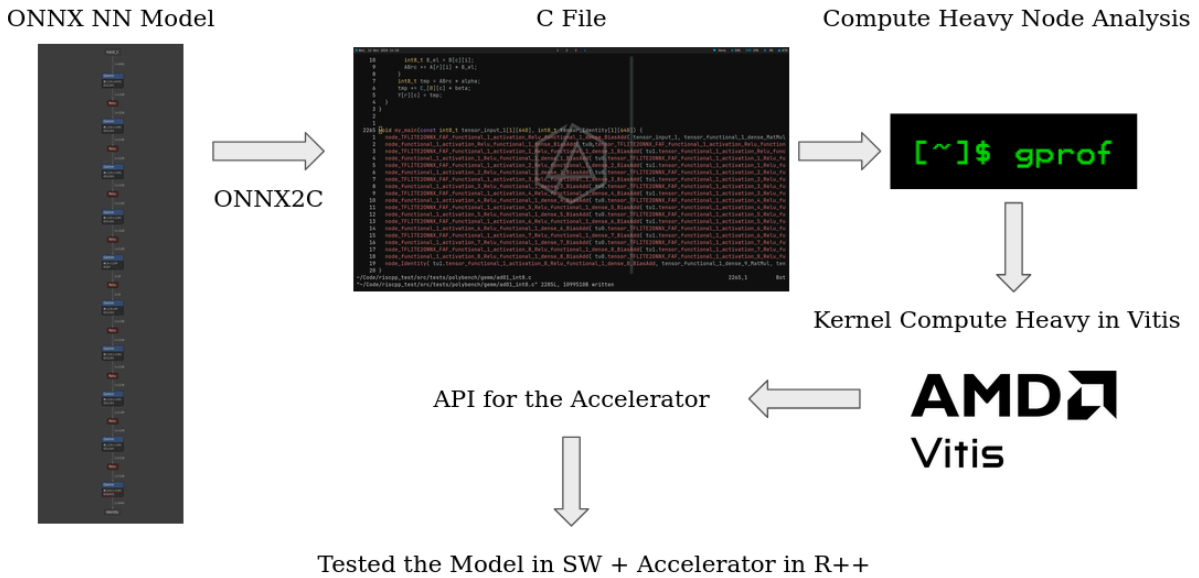
Input: ONNX model file (ad01_int8.onnx) representing a quantized neural network for anomaly detection in automotive audio signals.

Compilation: The onnx2c tool translates the ONNX graph into executable C code, preserving the computational structure while enabling profiling instrumentation.

Dataset Preparation: Audio samples from the ToyCar dataset are preprocessed using librosa to extract features suitable for the model input format.

Execution and Profiling: The compiled model processes the feature data while gprof collects performance metrics on individual node execution times.

Accelerator in RISC++ for compute heavy nodes in ONNX model



2.3 Test Models and Applications

The demonstration utilizes the `ad01_int8` model, an 8-bit quantized neural network designed for anomaly detection in machine audio. The model architecture includes:

- Convolutional layers for feature extraction
- Dense layers for classification
- Quantization operations optimized for edge deployment

The ToyCar dataset provides realistic audio samples recorded from functioning and malfunctioning toy car mechanisms, enabling evaluation of the model's computational characteristics under practical conditions.

2.4 Results Measurement

The `gprof` profiler generates a comprehensive performance report identifying the percentage of execution time consumed by each function (corresponding to ONNX nodes). This quantitative data enables prioritization of acceleration targets based on actual computational cost rather than architectural assumptions.

3. System Requirements

3.1 Software Dependencies

Component	Description
onnx2c	ONNX to C compiler with ProtocolBuffers dependencies
gprof	GNU profiler (included in binutils package)

Python 3	Runtime environment with librosa, numpy, tqdm libraries
ToyCar Dataset	Audio samples for model testing and profiling

3.2 Hardware Infrastructure

The demonstration requires a Linux or macOS environment with standard development tools (GCC compiler, make utilities). No specialized hardware is necessary for the profiling phase, though subsequent acceleration implementation would target specific platforms such as FPGAs or custom ASICs.

4. Installation and Setup Procedure

4.1 Installing onnx2c

Install ProtocolBuffers dependencies:

Ubuntu: `apt install libprotobuf-dev protobuf-compiler`

macOS: `brew install protobuf`

Clone and build onnx2c:

```
git clone https://github.com/kraiskil/onnx2c.git
cd onnx2c
git submodule update --init
mkdir build && cd build
cmake -DCMAKE_BUILD_TYPE=Release ..
make onnx2c
```

4.2 Installing gprof

```
sudo apt update
sudo apt install binutils
which gprof
gprof --version
```

4.3 Dataset Acquisition

```
chmod +x get_dataset.sh
./get_dataset.sh
```

5. Execution Workflow

5.1 Model Compilation

Convert the ONNX model to C code:

```
./onnx2c ad01_int8.onnx > ad01_int8.c
```

5.2 Data Preprocessing

Install Python dependencies and preprocess the dataset:

```
pip install librosa numpy tqdm
python3 preprocess.py --data_dir dev_data/ToyCar --out_dir
features_bin --train_dir dev_data/ToyCar/train
```

Note: The exact preprocessing pipeline from the original model training is not available. This preprocessing provides functionally equivalent features for profiling purposes, though it may affect model accuracy metrics. Since the objective is performance analysis rather than accuracy validation, this approximation is acceptable.

5.3 Benchmark Compilation and Execution

Compile the benchmark with profiling enabled:

```
gcc -pg -o benchmark benchmark.c ad01_int8.c -lm
./benchmark features.csv features_bin/stats.txt
```

5.4 Profile Analysis

Generate the profiling report:

```
gprof benchmark gmon.out
```

Or save to a file for detailed analysis:

```
gprof benchmark gmon.out > analysis.txt
```

6. Profiling Results and Analysis

6.1 Computational Hotspots

The gprof analysis of the ad01_int8 model execution reveals clear computational bottlenecks. The total execution time was 1.40 seconds for 2,459 inference calls, with the following operations consuming the majority of CPU time:

Operation	% Time	µs/call
node_Identity	37.86%	215.53
Dense Layer 0 + ReLU Activation	22.86%	130.13
Dense Layer 2 + ReLU Activation	8.57%	48.80
Dense Layer 6 + ReLU Activation	7.14%	40.67
Dense Layer 7 + ReLU Activation	7.14%	40.67
Other Dense Layers (1, 3, 8)	14.71%	Various

6.2 Key Findings

The profiling data reveals several critical insights for acceleration target selection:

Identity Operation Dominance: The node_Identity operation consumes 37.86% of total execution time (0.53 seconds), representing the single largest bottleneck. This

operation is called 2,459 times with an average cost of 215.53 microseconds per call. This suggests significant potential for optimization through specialized hardware implementation.

Dense Layer Operations: The combined dense layer computations (matrix multiplications with bias addition) and ReLU activations account for approximately 60% of the remaining execution time. The first dense layer (22.86%) shows significantly higher cost than subsequent layers, likely due to larger matrix dimensions or memory access patterns.

Acceleration Priority: Based on these measurements, hardware acceleration efforts should prioritize: (1) Identity operations and (2) Dense layer 0 computations. Optimizing these two operations would address over 60% of the computational workload.

6.3 Hardware Implementation Results

The profiling insights informed the development of two RISC-V based hardware implementations for the ad01_int8 model, demonstrating the impact of memory architecture on performance:

Configuration	Cycles	Resources	Clock	Speedup
RISC-V (BRAM) SW-only	2,028,973	3,361 LUTs1,558 FFs3 DSPs	50 MHz	11.0×
RISC-V (DDR) SW-only	22,318,703	4,217 LUTs4,572 FFs3 DSPs	35 MHz	1.0×

Performance Analysis: The BRAM-based implementation demonstrates an 11× cycle reduction compared to the DDR configuration (2.03M vs 22.32M cycles), while maintaining comparable instruction count (1.84M instructions). This dramatic performance difference highlights the critical impact of memory latency on neural network inference, particularly for the identity and dense layer operations identified in the profiling analysis.

Resource Utilization: The BRAM implementation achieves superior performance with fewer FPGA resources (3,361 vs 4,217 LUTs), demonstrating the efficiency of on-chip memory for this workload. The DDR configuration requires additional logic for memory controller interfaces, reflected in the increased flip-flop count (4,572 vs 1,558 FFs), while both implementations utilize identical DSP resources for arithmetic operations.

7. Limitations and Considerations

Quantization Parameters: The current benchmark implementation does not include complete quantization parameter handling. This affects accuracy metrics but does not impact the relative computational costs of different operations, which is the primary measurement objective.

Preprocessing Approximation: Without access to the original training pipeline, the preprocessing may not exactly match the model's expected input distribution. This is acceptable for profiling but should be addressed before deployment.

Platform Dependency: Profiling results reflect the characteristics of the host CPU architecture. While relative costs remain informative, absolute timing values should be interpreted in context of the target deployment platform.

8. Conclusion

This demonstration establishes a reproducible methodology for identifying acceleration targets within ONNX neural network models. By combining onnx2c compilation with gprof profiling, developers can make data-driven decisions about which operations to implement in hardware accelerators.

The approach supports the broader goal of efficient edge AI deployment by enabling systematic optimization of computational resources. Future work could extend this methodology to include power profiling, memory bandwidth analysis, and multi-platform comparison studies.

The quantitative insights generated through this workflow inform the design of application-specific accelerators, ensuring that hardware development efforts target the operations that provide maximum performance benefit for real-world workloads.